PROJECT REPORT

Online Shopper Purchase Prediction

An end-to-end machine learning project with a prediction API, a web interface and an analytics dashboard

Developer	Yasir Khan
Project type	Supervised binary classification, FastAPI service, Streamlit apps
Repository	github.com/YasirKhan2929/online-shopper-purchase-prediction
Report date	October 2026
Report version	1.0

Status. Educational portfolio project. The model is not claimed to be production-ready. Reported metrics are approximate and require reproducible verification. Items that could not be verified against the repository are labelled throughout (see Appendix A).

Table of Contents

1. Executive summary 3

2. Introduction 3

3. Problem statement 4

4. Project idea and objectives 4

5. Dataset description and data quality 5

6. Exploratory data analysis 6

7. Data preprocessing and feature engineering 7

8. Machine learning methodology 7

9. Model training and comparison 8

10. Threshold selection and evaluation 9

11. PageValues investigation and leakage considerations 11

12. Application architecture 12

13. API and user interface 13

14. Tools and technologies 14

15. Project structure 15

16. Setup and execution guide 16

17. Limitations and future improvements 17

18. Conclusion 18

19. References 18

Appendix A. Verification log 18

1. Executive summary

Online stores receive many visitor sessions, but most visitors leave without buying. This project builds a machine learning system that estimates, from the browsing-session information of a visitor, how likely that session is to end in a purchase. It covers the full path from data exploration to a saved model, a REST API, two Streamlit applications and a browser-based demo store.

The model is a scikit-learn Logistic Regression pipeline with preprocessing, trained on the public Online Shoppers Purchasing Intention dataset (12,330 sessions, 15.47% of which ended in a purchase). Predictions use a decision threshold of 0.60. On the held-out test set the project reports approximately 0.30 precision, 0.56 recall, 0.39 F1-score and 0.73 ROC-AUC. These figures show that the model finds useful signal but also produces many false positives, so it should be read as a learning and portfolio project rather than a deployable system.

A separate investigation of the *PageValues* feature showed a large validation improvement (ROC-AUC 0.740 to 0.908) when the feature was included. Because this feature may not be available at prediction time, or may partly reflect the outcome being predicted, the primary model excludes it. The API schema in the repository is consistent with this decision.

Item	Summary	Status
Dataset	Online Shoppers Purchasing Intention; 12,330 rows, 18 columns	**Reported**
Target	Revenue (purchase / no purchase); 1,908 purchases, 10,422 non-purchases	**Reported**
Model	Logistic Regression pipeline, saved with Joblib	**Reported**
Threshold	0.60	**Reported**
Test results	Precision approx. 0.30, recall 0.56, F1 0.39, ROC-AUC 0.73	**Reported**
Services	FastAPI (api.py), Streamlit (app.py, analytics.py), HTML demo store	**Verified**
Production readiness	Not claimed (README says the same)	**Verified**

How this report was verified. The repository README, `api.py` and `app.py` were read directly on GitHub. The `src/` scripts, `analytics.py`, `store_demo.html` and the screenshots folder could not be opened, so descriptions of those parts come from the project brief and file names and are labelled accordingly.

2. Introduction

This document describes the *Online Shopper Purchase Prediction* project: what problem it addresses, how the data was handled, how the model was built and evaluated, how it is served to users, and what its limitations are. It is intended for a GitHub portfolio, technical review and discussion in AI/ML interviews.

2.1 How to read the status labels

To keep the report honest about what was and was not checked, statements carry one of the following labels where it matters:

Label	Meaning
Verified	Confirmed by reading the repository files that could be opened (README, api.py, app.py, top-level file listing).
Reported	Taken from the project brief or README (for example evaluation figures) but not reproduced by running the code.
Dataset docs	Taken from the public documentation of the dataset, not from the repository.
Inferred	Reasoned from file names or from arithmetic on reported values; plausible but not confirmed in source.

Label	Meaning
Derived	Calculated in this report from reported numbers (for example metrics recomputed from the confusion matrix).
To confirm	A detail that must be checked in the source before being relied on.
Proposed	A suggested improvement that is not implemented.

3. Problem statement

3.1 The problem

Only a minority of website visits end in a purchase. A business that cannot tell which sessions are likely to convert has little evidence for deciding where to spend marketing effort, how to design pages, or when to offer help or incentives.

3.2 Why it matters

- **Marketing efficiency:** understanding which session patterns precede purchases helps target campaigns and reduce wasted spend.
- **Website design:** patterns such as long product-page browsing without purchase can point to friction in the buying journey.
- **Customer experience:** a probability score can, in principle, support timely and relevant interventions.

3.3 Proposed solution

The project frames the task as supervised binary classification. A model is trained on historical sessions labelled with whether a purchase occurred (`Revenue`) and outputs a purchase probability for a new session. A decision threshold converts that probability into a yes/no prediction.

3.4 Scope

In scope	Out of scope
Data exploration, preparation and model training on the public dataset	Collecting real customer traffic (the demo store uses illustrative values)
Threshold tuning and held-out evaluation	Production deployment, monitoring and autoscaling
A FastAPI prediction service and Streamlit interfaces	Authentication, user accounts, persistent session storage
Investigation of PageValues and leakage risk	Causal claims about why visitors buy

4. Project idea and objectives

The central idea is to take a standard tabular classification problem and carry it through the stages a real ML application needs: analysis, careful evaluation, a persisted model artefact, a service interface and user-facing apps.

4.1 Objectives

- Build a classifier that estimates purchase probability from session features.
- Evaluate it on held-out data with metrics suited to an imbalanced target, not accuracy alone.
- Select and document a classification threshold.
- Investigate whether a highly predictive feature (PageValues) is safe to use at prediction time.
- Expose the saved model through a validated REST API and make it usable through a form-based interface and a demo store.
- Provide an analytics dashboard that presents shopping-behaviour insights.

4.2 Deliverables

Deliverable	Location	Status
Numbered ML pipeline scripts (13)	src/	**Reported**
Saved model pipeline	models/purchase_prediction_model.joblib	**Reported**
Prediction API	api.py	**Verified**
Prediction interface	app.py	**Verified**
Analytics dashboard	analytics.py	**Reported**
Demo online store	store_demo.html	**Reported**

5. Dataset description and data quality

5.1 Overview

The project uses `online_shoppers_intention.csv`, the Online Shoppers Purchasing Intention dataset published through the UCI Machine Learning Repository (Sakar et al., 2019). Each row is one browsing session. The class counts below are given in both the README and the project brief.

Property	Value	Status
Rows (sessions)	12,330	**Verified**
Columns	18 (17 features plus the target)	**Reported**
Target column	Revenue	**Reported**
Purchase sessions	1,908	**Verified**
Non-purchase sessions	10,422	**Verified**
Purchase rate	15.47% (1,908 / 12,330)	**Verified**

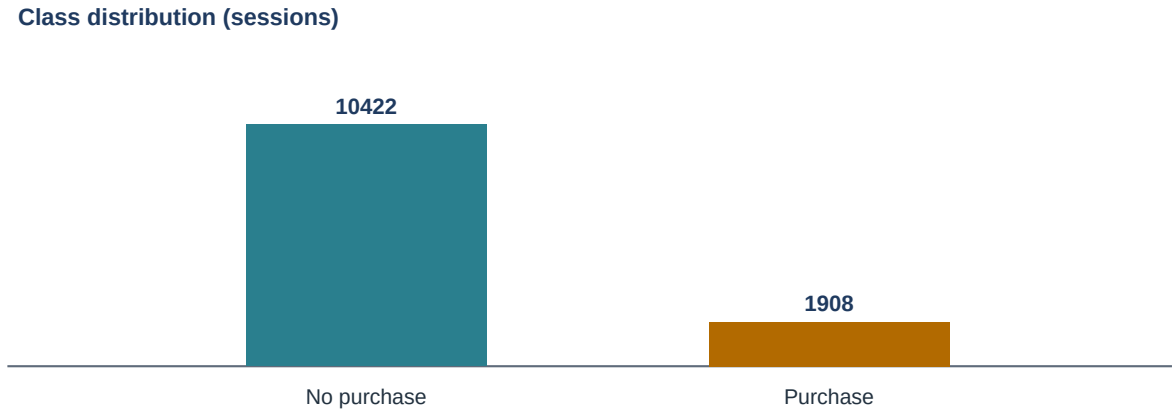


Figure 1. The target is imbalanced: purchases are about 15.5% of sessions.

5.2 Features

Column meanings below come from the public dataset documentation **Dataset docs**. The API type column is taken from the Pydantic model in `api.py`, which lists 16 input fields **Verified**. *PageValues* is the one dataset feature absent from that schema.

Feature	Meaning (dataset docs)	API type
Administrative	Number of administrative (account-type) pages visited	int >= 0
Administrative_Duration	Seconds spent on administrative pages	float >= 0
Informational	Number of informational pages visited	int >= 0

Feature	Meaning (dataset docs)	API type
Informational_Duration	Seconds spent on informational pages	float >= 0
ProductRelated	Number of product-related pages visited	int >= 0
ProductRelated_Duration	Seconds spent on product-related pages	float >= 0
BounceRates	Average bounce rate of the pages visited	float, 0 to 1
ExitRates	Average exit rate of the pages visited	float, 0 to 1
PageValues	Average page value of pages visited (see Section 11)	not in schema
SpecialDay	Closeness of the visit to a special day, scaled 0 to 1	float, 0 to 1
Month	Month of the visit	str
OperatingSystems	Integer code for the operating system	int
Browser	Integer code for the browser	int
Region	Integer code for the region	int
TrafficType	Integer code for the traffic source type	int
VisitorType	Returning_Visitor, New_Visitor or Other	str
Weekend	Whether the visit was on a weekend	bool
Revenue (target)	Whether the session ended in a purchase	n/a

5.3 Data quality checks

Topic	What is known	Status
Missing values	The saved pipeline includes missing-value handling. Whether the raw CSV actually contains missing values is not confirmed here.	**To confirm**
Duplicates	Whether duplicate sessions were checked or removed in the 01/03 scripts could not be verified. Public versions of this dataset are commonly reported to contain a small number of repeated rows.	**To confirm**
Class imbalance	Confirmed from the class counts above.	**Verified**
Month coverage	The Streamlit form offers ten months (Feb, Mar, May, June, Jul, Aug, Sep, Oct, Nov, Dec), consistent with the dataset not covering January and April.	**Verified**
Value ranges	The API enforces non-negative counts and durations and 0 to 1 for rates and SpecialDay.	**Verified**

5.4 Class imbalance

With roughly 1 purchase for every 5.5 non-purchases, a model that always predicts "no purchase" would already be right about 85% of the time while being useless for finding buyers. This is why evaluation in this project focuses on precision, recall, F1-score and ROC-AUC (Section 8.5), and why the decision threshold is tuned rather than left at the default of 0.5.

6. Exploratory data analysis

Two numbered scripts cover the exploratory phase. Their contents could not be opened, so their purposes are described from their names and the project brief.

Script	Purpose	Status
01_explore_data.py	First look at the dataset: shape, column types, sample rows, target balance, basic quality checks.	**Inferred**
02_analyze_data.py	Deeper analysis of how features relate to purchase outcome.	**Inferred**

The findings that can be stated with confidence are the overall class balance (Section 5.1) and the project's own observation that *PageValues* is strongly associated with purchase outcome (Section 11). The business-facing views of the data are presented in the Streamlit analytics dashboard (`analytics.py`); the README describes it as providing business insights.

> **To confirm before publishing:** list the specific EDA findings (for example purchase rate by visitor type, month and weekend) directly from the output of `02_analyze_data.py` so this section reports measured facts rather than descriptions of intent.

7. Data preprocessing and feature engineering

7.1 Prepared data

The README project tree lists a processed file, `data/processed/online_shoppers_prepared.csv`, which is produced by `03_prepare_data.py` **Inferred**. The saved model is a scikit-learn pipeline, so preprocessing is stored together with the estimator and is applied identically at training and prediction time.

7.2 Preprocessing steps

Step	Description	Status
Missing-value handling	Part of the saved pipeline.	**Reported**
Numerical scaling	Numerical features are scaled so that features with large ranges (such as durations in seconds) do not dominate a linear model.	**Reported**
Categorical encoding	Categorical features are encoded into numeric form for the model.	**Reported**
Exact column grouping	Which columns are treated as categorical (for example whether OperatingSystems, Browser, Region and TrafficType are one-hot encoded despite being integer-valued) and which encoder and imputer settings are used.	**To confirm**
Feature order	The saved file stores `feature_columns`; the API reorders incoming data to match it before predicting.	**Verified**

7.3 Train, validation and test data

Results are reported on a held-out test set, and the PageValues comparison is reported on validation data. The reported confusion matrix contains 2,466 sessions, which is exactly 20% of 12,330, suggesting an 80/20 train/test split **Inferred**. Whether the split was stratified by the target, the random seed, and how the validation set was formed are **To confirm**.

7.4 Feature engineering

No engineered (derived) features are confirmed. The API accepts the original dataset features only, minus *PageValues*, and the saved model works from those 16 inputs.

8. Machine learning methodology

8.1 Supervised binary classification

In supervised learning the algorithm learns from examples that already have the correct answer attached. Here each example is a session and the answer is purchase or no purchase. Because there are exactly two possible answers, this is binary classification.

8.2 Training, validation and testing

- **Training data** is used to fit the model's parameters.

- **Validation data** is used to compare models and choose settings such as the threshold, without touching the final test set.
- **Test data** is held back until the end and gives an estimate of performance on sessions the model has never seen. Using it repeatedly to make choices would make the estimate optimistic.

8.3 Logistic Regression

Logistic Regression computes a weighted sum of the (preprocessed) features and passes it through the logistic function, giving a number between 0 and 1: $p = 1 / (1 + e^{-z})$, where $z = b_0 + b_1x_1 + b_2x_2 + ...$ The result is interpreted as the estimated probability of purchase. It is fast, stable and easy to inspect, which makes it a sensible baseline and a good model for explaining results.

8.4 Probability scores and thresholds

The model outputs a probability, not a decision. A threshold turns it into a decision: if the probability is at least the threshold, the session is predicted as a purchase. Raising the threshold makes the model more cautious (fewer false positives, more missed buyers); lowering it does the opposite. This project uses 0.60 **Reported**; the API reads the active value from the saved model file and reports it at `/health`.

8.5 Evaluation metrics

Metric	Plain meaning	Formula
Accuracy	Share of all predictions that are correct	(TP + TN) / all sessions
Precision	Of the sessions predicted as purchases, how many really were	TP / (TP + FP)
Recall	Of the real purchases, how many the model found	TP / (TP + FN)
F1-score	Single number balancing precision and recall (harmonic mean)	2PR / (P + R)
ROC-AUC	How well the model ranks buyers above non-buyers across all thresholds; 0.5 is chance, 1.0 is perfect	Area under the ROC curve

TP, TN, FP and FN stand for true positives, true negatives, false positives and false negatives.

8.6 Why accuracy alone can mislead

On an imbalanced dataset, accuracy rewards predicting the majority class. In the test set described in Section 10, always predicting "no purchase" would give about 84.5% accuracy (derived), higher than the model's own accuracy of about 72.9% (derived), even though it would find no buyers at all. The model is judged by how well it finds buyers and at what cost in false alarms.

8.7 Saving and reusing the pipeline

`08_save_model.py` is described in the brief as the model-saving step. From `api.py` it is confirmed that the saved file is a dictionary with the keys `model`, `threshold` and `feature_columns` **Verified**. The API loads it once at start-up and reuses it for every request, so preprocessing and prediction cannot drift apart.

> **Not production-ready.** The model has modest precision, has not been cross-validated in the material reviewed, and has no automated tests or monitoring. It demonstrates the workflow, not a deployable product.

9. Model training and comparison

The project is organised as 13 numbered scripts in `src/`, meant to be read and run in order. The script names and numbering come from the project brief and the README project tree **Reported**; the purposes below are inferred from the names because the files were not opened.

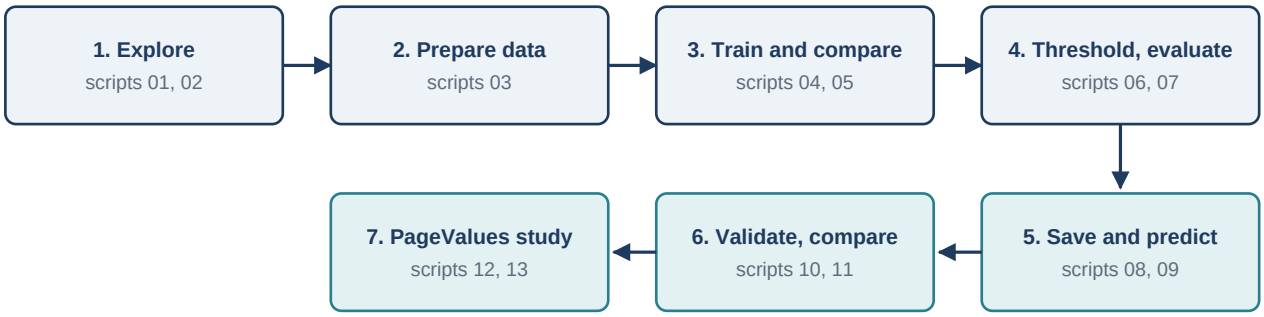


Figure 2. Development workflow grouped into seven phases.

#	Script	Purpose	Why it matters
1	01_explore_data.py	Initial dataset exploration	Understand shape, types and quality before modelling
2	02_analyze_data.py	Data analysis	Find which behaviours relate to purchases
3	03_prepare_data.py	Data preparation	Produce clean input (prepared CSV listed in README)
4	04_train_model.py	Model training	Fit the first working classifier
5	05_compare_models.py	Model comparison	Check the choice against alternatives
6	06_tune_threshold.py	Classification threshold tuning	Balance precision and recall on imbalanced data
7	07_final_evaluation.py	Final evaluation	Report performance on held-out test data
8	08_save_model.py	Model saving	Persist the pipeline for reuse by the API
9	09_predict.py	Making predictions	Demonstrate inference with the saved pipeline
10	10_compare_validation.py	Validation comparison	Compare candidates on validation data
11	11_final_model_comparison.py	Final model comparison	Confirm the final model choice
12	12_investigate_pagevalues.py	Investigate PageValues	Understand an unusually strong feature
13	13_compare_pagevalues.py	Compare with and without PageValues	Quantify the effect on performance

All purposes in this table are **Inferred** from file names.

9.1 Model selection

The saved and served model is a Logistic Regression pipeline **Reported**. Scripts 05 and 11 compare models, but which alternative algorithms were tried and their scores are **To confirm** and are deliberately not stated here. No improved scores are claimed beyond those reported in Section 10.

10. Threshold selection and evaluation

10.1 Held-out test results at threshold 0.60

The README lists these as previously recorded held-out test results and describes them as approximate **Reported**.

Metric	Reported value
Precision	approx. 0.30
Recall	approx. 0.56
F1-score	approx. 0.39
ROC-AUC	approx. 0.73

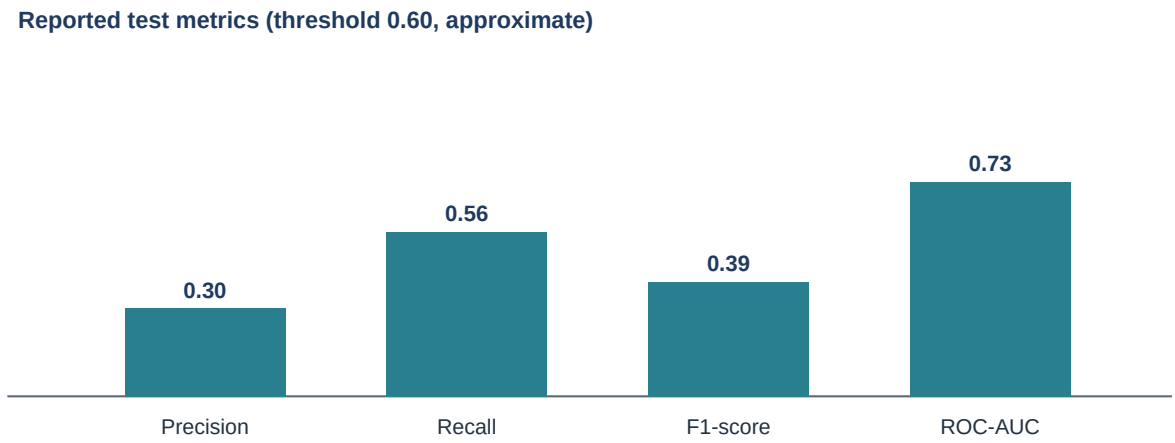


Figure 3. Reported held-out test metrics.

10.2 Confusion matrix

Approximate confusion matrix from the project brief: [[1582, 502], [167, 215]] **Reported**.

	Predicted: no purchase	Predicted: purchase
Actual: no purchase	1,582 (true negatives)	502 (false positives)
Actual: purchase	167 (false negatives)	215 (true positives)

10.3 Metrics recomputed from the confusion matrix

Recomputing the metrics from the matrix is a consistency check on the reported figures **Derived**.

Quantity	Calculation	Result
Test sessions	1582 + 502 + 167 + 215	2,466
Actual purchases in test set	167 + 215 (15.5% of test set)	382
Precision	215 / (215 + 502)	0.300
Recall	215 / (215 + 167)	0.563
F1-score	2PR / (P + R)	0.391
Accuracy	(1582 + 215) / 2466	0.729
Specificity (true negative rate)	1582 / (1582 + 502)	0.759
Always-"no purchase" accuracy	(1582 + 502) / 2466	0.845

The recomputed precision, recall and F1-score agree with the reported 0.30, 0.56 and 0.39, and the test purchase rate matches the dataset's 15.5%. This supports the internal consistency of the figures; it does not replace re-running `07_final_evaluation.py` to reproduce them.

10.4 Interpreting the trade-offs

- **Precision 0.30:** about 3 in 10 sessions flagged as likely purchases are real purchases; 7 in 10 are false alarms.
- **Recall 0.56:** the model catches a little over half of the sessions that actually end in a purchase and misses the rest.
- **ROC-AUC 0.73:** a randomly chosen purchasing session receives a higher score than a randomly chosen non-purchasing one about 73% of the time. That is better than chance but only a moderate ability to rank sessions.
- **False positives** cost effort or money if the business acts on them, for example by showing discounts to people who would have bought anyway. **False negatives** are missed opportunities, for example failing to help a hesitant buyer. Which error is more expensive depends on the intervention, so the threshold should be revisited whenever those costs are known.

10.5 Threshold choice

Moving the threshold from the default 0.5 to 0.60 trades some recall for higher precision. The threshold-tuning step (`06_tune_threshold.py`) and the rationale for selecting 0.60 specifically, including the criterion used (for example F1-score on validation data), are **To confirm** from the script.

11. PageValues investigation and leakage considerations

11.1 What PageValues represents

According to the public dataset documentation, *PageValues* is the average value of the pages a visitor viewed before completing an e-commerce transaction, a metric derived from web analytics **Dataset docs**. By construction it is calculated using transaction information, which is closely tied to whether a purchase happens.

11.2 What the project found

The project compared models with and without the feature. These are **validation results, not held-out test results** **Reported**.

Setting	ROC-AUC	F1-score
Without PageValues	0.740	0.378
With PageValues	0.908	0.609
Difference **Derived**	+0.168	+0.231

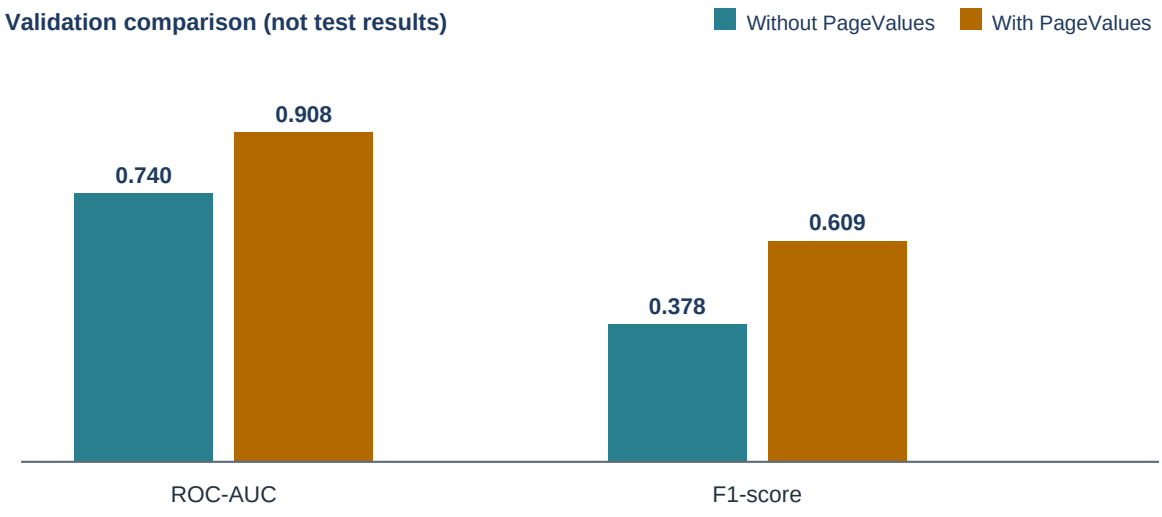


Figure 4. Validation ROC-AUC and F1-score with and without PageValues.

11.3 Why a very strong feature can be a problem

- **Correlation is not causation.** A feature can be strongly associated with purchases without causing them. Page value is high on pages that tend to appear in sessions that end in purchases, which is not the same as those pages making visitors buy.
- **Target leakage.** Leakage happens when a feature contains information that would not be known at the moment a prediction is needed, or that is partly computed from the outcome. A model using such a feature looks excellent in testing and disappoints in real use.
- **Availability at inference.** A purchase-prediction system is useful while the visitor is still browsing. If PageValues can only be computed after, or with knowledge of, the transaction, it cannot be supplied at that time.

11.4 Decision taken and its evidence

The primary model does not use *PageValues*. This is consistent with the repository: the `VisitorSession` schema in `api.py` has 16 fields and no PageValues field **Verified**. The feature should only be considered if it can be shown that its value is available, and calculated without the outcome, at prediction time.

> **Careful wording.** This investigation does not conclusively prove leakage. It identifies a serious leakage and availability risk that justifies excluding the feature until it is resolved. The without-PageValues figures (ROC-AUC 0.740, F1 0.378) are the realistic validation reference for this project.

12. Application architecture

The application is a small service-oriented system. The model is loaded once by the FastAPI service. Two user interfaces send session data to it and display the result.

Solid arrows: requests. Dashed arrows: responses.

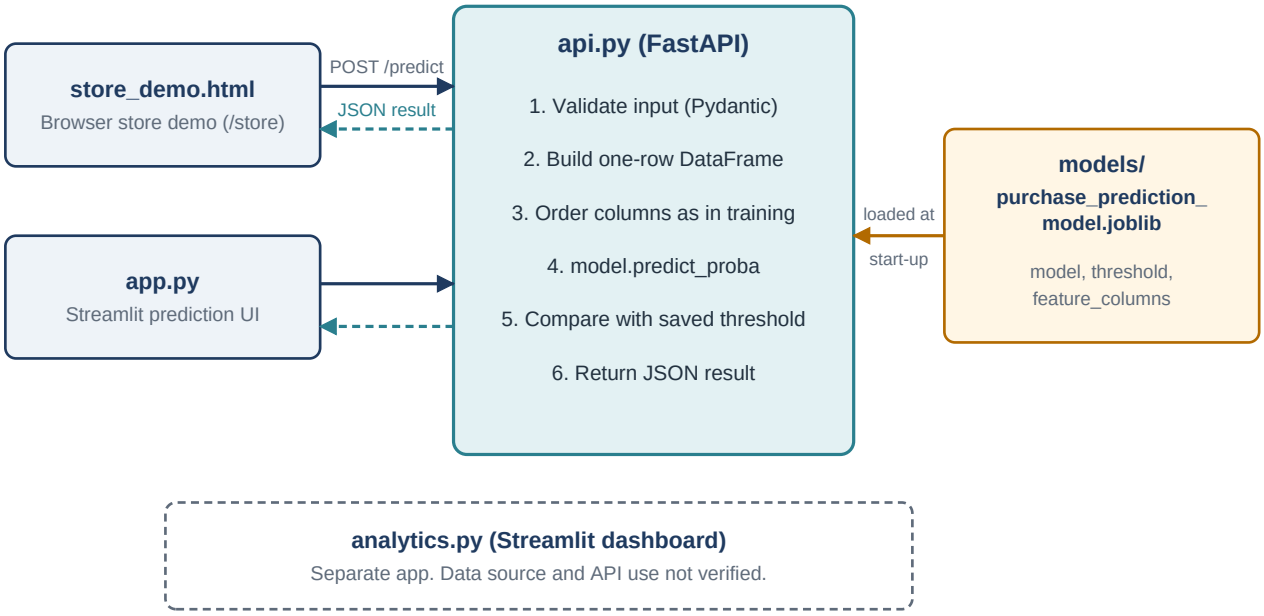


Figure 5. Application architecture.

12.1 Components

Component	Role	Status
api.py	FastAPI service (123 lines). Loads the saved model, validates input, predicts, serves the demo store.	**Verified**
app.py	Streamlit form (206 lines). Collects session details, posts them to the API, shows the probability and decision.	**Verified**
analytics.py	Streamlit business analytics dashboard.	**Reported**
store_demo.html	Browser demo store sending illustrative session data to the API; README states the values are not real customer analytics.	**Reported**
models/purchase_prediction_model.joblib	Saved pipeline, decision threshold and feature list.	**Verified**

12.2 Request flow

- The user enters session details in the Streamlit form or the demo store.
- The interface sends a JSON request to `POST /predict`.
- FastAPI validates the body against the `VisitorSession` model; invalid input is rejected.
- The data becomes a one-row DataFrame, ordered to match `feature_columns`.
- The pipeline's `predict_proba` returns the purchase probability.
- The probability is compared with the saved threshold to give a decision.
- The JSON response returns to the interface, which displays it.

13. API and user interface

13.1 Endpoints

The following is documented from `api.py` **Verified**.

Endpoint	Behaviour
GET /	Returns a message that the API is running plus pointers to /docs and POST /predict.
GET /health	Returns status, whether the model is loaded, and the active threshold.
POST /predict	Validates a visitor session and returns the purchase prediction (details below).
GET /store	Serves store_demo.html; returns 404 if the file is missing. Hidden from the OpenAPI schema.
/docs	Interactive API documentation generated automatically by FastAPI.

13.2 Request fields for POST /predict

Field	Type	Constraint in schema
Administrative, Informational, ProductRelated	int	greater than or equal to 0
Administrative_Duration, Informational_Duration, ProductRelated_Duration	float	greater than or equal to 0
BounceRates, ExitRates, SpecialDay	float	0 to 1 inclusive
Month, VisitorType	str	none in the schema
OperatingSystems, Browser, Region, TrafficType	int	none in the schema
Weekend	bool	true or false

13.3 Response and errors

Case	Response
Success (200)	purchase_probability, purchase_probability_percent, threshold, prediction ("Purchase likely" or "Purchase unlikely"), predicted_purchase (boolean), note
Input does not match expected features (KeyError, ValueError)	HTTP 422 with an explanatory detail message
Schema violations (wrong type or out-of-range value)	HTTP 422, produced automatically by FastAPI and Pydantic
Any other failure	HTTP 500 with a generic message; details appear in the server terminal

Observation. *Month* and *VisitorType* are free-text strings in the schema. How the pipeline treats a value it has never seen (for example a misspelt month) depends on how the encoder was configured and should be tested **To confirm**.

13.4 Example request

Constructed from the default values in the Streamlit form in `app.py`. It is illustrative; no response value is shown because the API was not run for this report.

```
{
 "Administrative": 0, "Administrative_Duration": 0.0,
 "Informational": 0, "Informational_Duration": 0.0,
 "ProductRelated": 5, "ProductRelated_Duration": 120.0,
 "BounceRates": 0.02, "ExitRates": 0.03, "SpecialDay": 0.0,
 "Month": "Feb", "OperatingSystems": 1, "Browser": 1,
 "Region": 1, "TrafficType": 1,
 "VisitorType": "Returning_Visitor", "Weekend": false
}
```

13.5 Streamlit prediction interface

`app.py` presents a two-column form with the 16 inputs, a Predict button, and a result area showing the probability as a percentage and a progress bar, a success or warning message, and the threshold used. It calls `http://127.0.0.1:8000/predict` with a 15-second timeout. It handles connection errors (with a hint to start the API), timeouts, HTTP errors (showing the API's detail text) and unexpected responses **Verified**. It also states that predictions are estimates and should not be the sole basis for business decisions.

13.6 Analytics dashboard and demo store

The analytics dashboard (`analytics.py`) and demo store (`store_demo.html`) were not opened. Per the README the dashboard presents business insights and the store posts illustrative browsing-session data to the prediction API. The README port note (use `--server.port 8502` if 8501 is busy) applies to the dashboard.

> **Screenshots.** The repository has a `screenshots/` folder, but it could not be accessed in this review, so no screenshots are embedded. Add images of the prediction form, the demo store and the dashboard here before sharing the report externally.

14. Tools and technologies

The table lists what the project uses. Status shows how each tool's use was confirmed; nothing here is listed as implemented unless the repository or project information supports it.

Tool	Role in the project	Status
Python	Language for all scripts, the API and the apps	Verified
pandas	Tabular data handling; builds the one-row input in api.py	Verified
NumPy	Numerical computing (README stack)	Reported
scikit-learn	Pipeline, preprocessing and Logistic Regression	Reported
Joblib	Saves and loads the model pipeline (api.py uses joblib.load)	Verified
FastAPI	REST API with automatic validation and /docs	Verified
Pydantic	Input schema (VisitorSession); installed with FastAPI	Verified
Uvicorn	Server that runs the FastAPI app	Reported
Streamlit	Prediction interface and analytics dashboard	Verified
Requests	Streamlit app calls the API with requests.post	Verified
Matplotlib	Plotting; listed in README stack and install command	Reported
HTML	Demo store page (store_demo.html)	Reported
Git and GitHub	Version control and hosting (4 commits visible)	Verified
Visual Studio Code	Development environment; README refers to VS Code terminals	Reported
PowerShell	Windows shell for setup and running; README uses PowerShell activation	Reported

Tools such as pytest, experiment trackers, containers or cloud hosting are not part of the implemented project; they appear only as proposals in Section 17.

15. Project structure

The top-level GitHub listing shows the entries below. The README tree additionally lists `notebooks/` and `tests/`.

Path	Purpose	On GitHub top level
api.py	FastAPI prediction service	Yes
app.py	Streamlit prediction interface	Yes
analytics.py	Streamlit analytics dashboard	Yes
store_demo.html	Browser demo store	Yes
README.md	Project overview, install and run guide	Yes
.gitignore	Files excluded from version control	Yes
data/raw/	online_shoppers_intention.csv (per README)	data/ present; contents not opened
data/processed/	online_shoppers_prepared.csv (per README)	data/ present; contents not opened
models/	purchase_prediction_model.joblib (per README)	models/ present; contents not opened
src/	Scripts 01 to 13	Yes; contents not opened
screenshots/	Project screenshots	Yes; contents not opened
notebooks/	Notebooks (listed in README tree)	Not shown on GitHub
tests/	Tests (listed in README tree)	Not shown on GitHub

> **No tests are implemented.** The `tests/` directory appears in the README tree but not in the GitHub listing. Git does not store empty folders, so these directories likely exist locally but are empty. The README itself lists automated tests as future work. The data and model files should be checked: the README tells users to place the CSV and ensure the model file is available, which may mean they are excluded by `.gitignore`.

16. Setup and execution guide

These Windows PowerShell steps follow the project brief and the README. Open PowerShell or a Visual Studio Code terminal.

Step 1. Go to the project folder

The path contains spaces, so it must be wrapped in quotation marks.

```
cd "C:\Users\PMYLS\Downloads\journy sept 2026\AI_ML_journey\project1"
```

Step 2. (Optional) Create and activate a virtual environment

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

If PowerShell refuses to run the activation script because of its execution policy, allow scripts for the current user with `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` and try again.

Step 3. Install dependencies

```
python -m pip install pandas numpy scikit-learn joblib fastapi "uvicorn[standard]"
streamlit requests
```

The README's install command also includes `matplotlib`. If the dashboard reports a missing module, run `python -m pip install matplotlib`, or add it to the command above.

Step 4. Start the prediction API (Terminal 1)

```
python -m uvicorn api:app --reload
```

Address	What it shows
http://127.0.0.1:8000/docs	Interactive API documentation
http://127.0.0.1:8000/health	Health check with model status and threshold
http://127.0.0.1:8000/store	Browser demo store

Step 5. Start the prediction interface (Terminal 2)

```
python -m streamlit run app.py
```

Important: the API must keep running in Terminal 1 while the Streamlit interface is used, because the interface sends its requests to the local API. If it is not running, the interface shows a connection error.

Step 6. Start the analytics dashboard (Terminal 3, optional)

```
python -m streamlit run analytics.py
# if port 8501 is already in use:
python -m streamlit run analytics.py --server.port 8502
```

Running the numbered scripts

To reproduce the pipeline, run the scripts in numeric order from the project folder, for example `python src\01_explore_data.py` **To confirm** (check each script's expected working directory and file paths). The raw CSV must be at `data\raw\online_shoppers_intention.csv`.

17. Limitations and future improvements

17.1 Limitations

Area	Limitation
Imbalanced classes	Only about 15.5% of sessions are purchases; the model must be judged with precision, recall and F1, not accuracy.
Error trade-off	Precision of about 0.30 means most positive predictions are false alarms; recall of 0.56 means many buyers are missed.
Reported metrics	Figures are approximate and have not been independently reproduced in this review.
Leakage and availability	PageValues is excluded because of leakage and availability risk, which is a concern to resolve rather than a proven fact. Other features should be checked for availability at prediction time too.
Generalisation	The data is a fixed historical sample from one store; new visitors, seasons and campaigns may behave differently.
Input validation	Month and VisitorType accept any string; integer codes have no bounds in the schema.
Testing and monitoring	No automated tests, drift monitoring or logging of predictions.
Reproducibility	Split method, seeds and package versions are not pinned in the material reviewed.
Privacy and responsible use	Session data can relate to identifiable people. Real deployment would need consent, minimisation, retention limits and care that predictions are not used to treat visitors unfairly.

17.2 Future improvements

Improvement	Benefit	Status
Automated tests for API endpoints and validation	Catch regressions; listed in README	**Proposed**
Compare additional classification models	Possible better ranking; listed in README	**Proposed**
Model performance monitoring	Detect drift after deployment; listed in README	**Proposed**
Persistent session storage for the demo store	Closer to a real data flow; listed in README	**Proposed**
Deploy the API and dashboards	Shareable live demo; listed in README	**Proposed**
Cross-validation	More reliable performance estimates	**Proposed**
Stronger comparisons (for example gradient-boosted trees)	Test whether non-linear models help	**Proposed**
Probability calibration	Make probabilities trustworthy as probabilities	**Proposed**
Explainability (coefficients, SHAP)	Explain individual predictions and feature effects	**Proposed**
Experiment tracking and pinned dependencies	Reproducible runs and results	**Proposed**
Stricter input validation (enums for Month and VisitorType)	Clear errors for invalid categories	**Proposed**
Improved interface design	Clearer results and explanations	**Proposed**

18. Conclusion

This project takes a public purchase-intention dataset from raw data through to a working prediction service and two Streamlit applications. It handles an imbalanced target with appropriate metrics, tunes a decision threshold, and, importantly, questions a feature that looked too good to be true rather than using it to inflate scores.

The reported performance is modest (approximately 0.30 precision, 0.56 recall, 0.73 ROC-AUC), and the model is not production-ready. Its value is as a clear, honest demonstration of the ML workflow and of engineering judgement: separating validation from test results, documenting limitations and keeping a suspect feature out of the primary model. The most valuable next steps are to reproduce the metrics, add automated tests and cross-validation, and verify the items marked *To confirm* in Appendix A.

19. References

[1] Sakar, C. O., Polat, S. O., Katircioglu, M., and Kastro, Y. (2019). Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks. *Neural Computing and Applications*, 31, 6893-6908.

[2] Online Shoppers Purchasing Intention Dataset. UCI Machine Learning Repository. https://archive.ics.uci.edu

[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

[4] FastAPI documentation. https://fastapi.tiangolo.com

[5] Streamlit documentation. https://docs.streamlit.io

[6] Khan, Y. Online Shopper Purchase Prediction (source repository). https://github.com/YasirKhan2929/online-shopper-purchase-prediction

Appendix A. Verification log

What was checked directly, what was not, and what should be confirmed in the source before this report is shared.

Item	Result	Status
README (overview, structure, install, run, evaluation)	Read in full on GitHub	**Verified**
api.py (endpoints, schema, errors, saved-file keys)	Read in full; documented in Sections 12 and 13	**Verified**
app.py (form, API call, error handling)	Read in full	**Verified**
Top-level repository listing	data, models, screenshots, src, five files plus .gitignore and README	**Verified**
src/ scripts 01 to 13	Folder blocked to automated access; purposes inferred from file names	**To confirm**
analytics.py, store_demo.html	Not opened; described from README	**To confirm**
screenshots/	Not accessible; none embedded	**To confirm**
Evaluation metrics and confusion matrix	From brief and README; internally consistent (Section 10.3); not re-run	**Reported**
PageValues validation numbers	From brief; labelled validation, not test	**Reported**
Duplicates, missing values, split method, encoder settings	Not confirmed	**To confirm**

Item	Result	Status
Alternative models compared in scripts 05 and 11	Not confirmed; no scores stated	**To confirm**
Whether data/ and models/ files are committed	README suggests they may be excluded	**To confirm**